Department of Electrical Engineering

ASIC/FPGA Chip Design

Lab 6 - Communication between PS and PL through BRAM

Contents

Lab 6 - Communication between PS and PL through BRAM

1.1 Objective

In this laboratory exercise, you will gain practical experience utilizing Block RAMs (BRAMs) available on the development board to facilitate efficient data transfer between the Processing System (PS) and the Programmable Logic (PL). Upon completion of this lab, you will be able to implement a functional signal generator, leveraging the data generation capabilities of the ARM processor and the signal generation potential of the Artix FPGA fabric.

We will also introduce a debugging interface for your hardware known as Integrated Logic Analyzer (ILA).

1.2 Introduction

On the Xilinx Zynq-7010, the Block RAMs (BRAMs) serve as a shared memory resource that can be accessed by both the Processing System (PS) and the Programmable Logic (PL). This enables efficient data exchange between software running on the PS and hardware accelerators/custom logic in the PL.

1.2.1 How BRAM Works for PS-PL Communication

- Dual-Port Memory Structure:
  - Each BRAM can be configured as a dual-port memory, allowing simultaneous access from both PS and PL.
  - The PS accesses BRAM via the AXI BRAM Controller (memory-mapped AXI interface).
  - The PL accesses BRAM directly via its dedicated ports.
- Memory Mapping:
  - The PS sees the BRAM as a region in its physical memory space (e.g., 0x4000_0000 to 0x4001_FFFF depending on configuration).
  - The PL can read/write the same memory locations independently.

1.2.1.1 Use Cases and Suitable Data Ranges

BRAM is best suited for small to medium-sized data transfers where low latency and deterministic access are critical.

1.2.2 Harnessing the Integrated Logic Analyzer (ILA) for Circuit Debugging

Highly complex digital circuits often present intricate debugging challenges. Traditional methods can be time-consuming and may not provide the necessary visibility into the dynamic behavior of the hardware. Here, the Integrated Logic Analyzer (ILA) emerges as an indispensable tool. The ILA allows us to capture and observe internal signals within the FPGA in real-time, providing crucial insights into circuit operation and enabling efficient identification and resolution of design flaws. Hence, in this lab, we will specifically utilize the System ILA, a powerful on-chip debugging core, to directly monitor and analyze the activity on our AXI buses. This hands-on experience will equip you with essential skills for effectively debugging complex hardware systems.

⚠ Important Notes

- Make sure to watch videos number 11.a, 11.b, 11.c, 11.d, 13 from Zynq Tutorial Videos prior to the lab session. To fully understand the flow of material, it is highly recommended to watch videos 12.a and 12.b.
- Study RAM architecture carefully, so that its notations seem intuitive to you.
- Instructions in this document will get you through all the necessary steps to construct and debug a complete signal generator. Should you feel confident enough, you may skip to the last section from the start!

1.3 Instructions

1.3.1 Text Ciphers

To initiate the exploration of block memory interactions, we begin with a foundational case study.

1. The Processing System (PS) will first encrypt the string "ILOVEZYNQ" using a Caesar Cipher.
2. The encrypted string will then be stored at an arbitrary address within a Block RAM (BRAM).
3. The Programmable Logic (PL) retrieves the encrypted string from the specified address.
4. PL decrypts the string and writes the result back to the BRAM at a predetermined destination address.
5. Finally, PS retrieves the decrypted string and performs an error-checking comparison between the original and decrypted strings.

As previously discussed, both PS and PL can interact with BRAM through read and write operations on designated address lines. PS executes these operations using the AXI BRAM Controller IP, while PL directly interfaces with the Block Memory Generator ports.

However, three key ambiguities arise in the described scenario:

- How does PL determine the starting address of the encrypted string?
- How is the length of the string communicated to PL?
- How does PS ascertain the destination address of the decrypted string?

These considerations necessitate a predefined protocol between PL and PS. Given that PS orchestrates the entire process, it is reasonable to assume that PS will provide PL with the requisite parameters: the starting address, string length, and destination address.

1.3.1.1 Custom IP Core

To address these challenges, we have developed a custom IP core named *pl_ps_bram_interconnect*. This IP features the following input/output ports:

- `BRAM_PORTB`
- `S00_AXI`
- `write_end`

The `BRAM_PORTB` interface facilitates read and write operations with the BRAM IP.

The IP core incorporates five 32-bit registers, initialized by the PS via the AXI protocol. These registers, referred to as slave registers, are entirely managed by the PS, hence the IP's designation as *pl_ps_bram_interconnect*.

The single-bit `write_end` signal is asserted by PL to notify PS that all read and write operations to BRAM have been completed. The overall system architecture is illustrated in Figure 1.1.

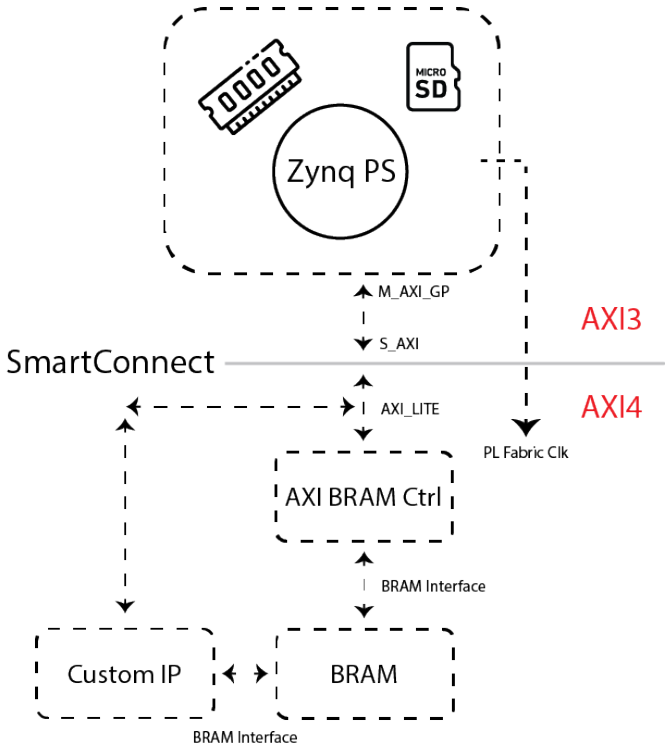

Figure 1.1: System architecture of the proposed design

1.3.1.2 Experimental Procedure

1. Integrate the IP core into your project repository and modify it using the IP Packager.
2. Examine the ***ram_read_write*** module, which implements a finite state machine (FSM). At each iteration, the IP reads a 4-byte word from BRAM, writes a word, and increments the address. The FSM states are defined as follows:
   (a) `IDLE`: Waits for the slave register *start* signal to be asserted. Upon activation, enables receive mode and initializes the address.
   (b) `DECOY`: Accounts for the one-cycle latency in the BRAM read operation; data becomes available one clock cycle after address assignment.
   (c) `READ_RAM`: Stores the retrieved BRAM data (4 bytes) into a register.
   (d) `WRITE_RAM`: Configures the write address and outputs the data.

(e) `ADDR_FETCH`: Advances to the next address. If the specified *length* of words has been processed, transitions to the `DONE` state.

(f) `DONE`: Asserts the *write_end* signal and returns to the `IDLE` state.

3. Finalize the *ram_read_write* module. Verify its functionality through simulation and ensure all state transitions are correctly implemented.
4. Repackage the IP and return to the main project.
5. Construct a block design matching the schematic in Figure 1.1.
6. Generate the bitstream and proceed to the SDK environment.
7. Utilize the provided `example.c` template to develop the cipher application. Much of the necessary code to work with the slave registers and the BRAM IP from PS side can be readily integrated in your application project.
8. Assess system performance by quantifying the error rate.

⚠ Critical Considerations

- BRAM write operations are non-blocking; confirmation of completion is unnecessary once the address and data lines are set.
- Make sure to configure the Block Memory Generator IP with "True Dual-Port" Option. Also, from its "Other Options" tab, uncheck "Enable Safety Circuit" as there is no need for parity checking and evaluation of data integrity. These two options can be viewed in Figure 1.2.
- The BRAM IP employs byte addressing but is word-aligned. Each address references a single byte, but a read operation fetches four contiguous bytes (one word).
- After constructing the block design, ensure the address range of each peripheral, particularly the AXI BRAM Controller, is properly configured in the PS address space. Allocate sufficient memory to accommodate the required data.
- You may disable the DDR chip in the PS configuration completely as it will not be utilized in any section of this lab exercise.
- Make sure to enable UART from the PS peripheral devices as you are to interactively communicate with the user in a serial monitor environment.
- Currently, no direct mechanism exists for PS to detect the PL *write_end* signal. As an interim solution, introduce deliberate delays in the PS C code using idle loops and the `asm("nop")` assembly instruction.

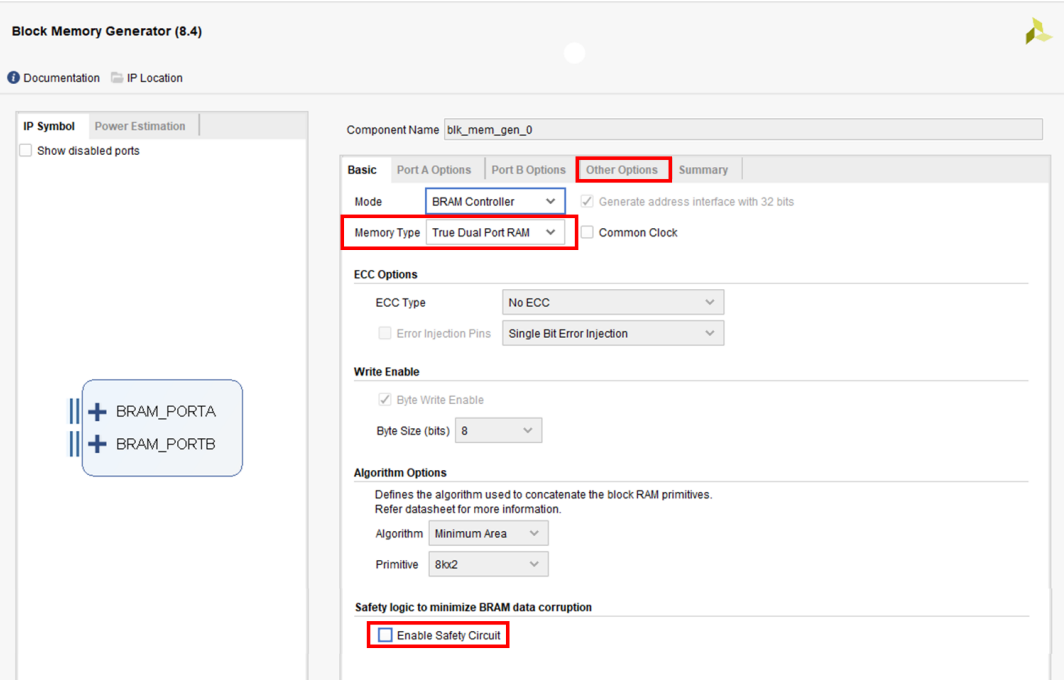

Figure 1.2: Block Memory Generator configurations

1.3.2 Sine Wave Generation

1.3.2.1 Direct Digital Synthesis

Modern signal generators employ digital logic to produce precise waveforms through a technique known as Direct Digital Synthesis (DDS). Consider an array containing samples of an arbitrary waveform over one period. By sequentially outputting these samples with programmable time intervals, a system can generate signals at arbitrary frequencies through resampling.

For periodic waveforms such as sine waves, the Processing System (PS) can leverage its computational capabilities for mathematical operations and scalar computations. However, more computationally intensive tasks like sample array resampling are better suited for implementation in the Programmable Logic (PL) fabric. This is where Block RAM (BRAM) becomes instrumental in achieving high-performance signal generation.

1.3.2.2 System Integrated Logic Analyzer (ILA)

The System Integrated Logic Analyzer (ILA) is an Xilinx IP core designed for real-time debugging of AXI datapaths. When a user-defined trigger condition occurs, the ILA captures and stores the values of all monitored ports for a configurable duration following the trigger event. The maximum acquisition depth for this IP core is 1024 samples.

1.3.2.3 Experimental Procedure

The experimental workflow consists of the following steps:

- The PS generates base-frequency sine wave samples using mathematical functions
- These samples are stored contiguously in BRAM
- The PL resamples these values to produce higher frequency waveforms
- The processed samples are written back to BRAM at a designated address
- The PS retrieves and visualizes the resulting waveform

The implementation must support frequency multiplication factors of 1×, 2×, 4×, and 8× relative to the original waveform. These configuration parameters are stored in the custom IP's slave registers, which are controlled via the PS master general-purpose (GP) port.

The overall block design of this section is provided in Figure 1.3.

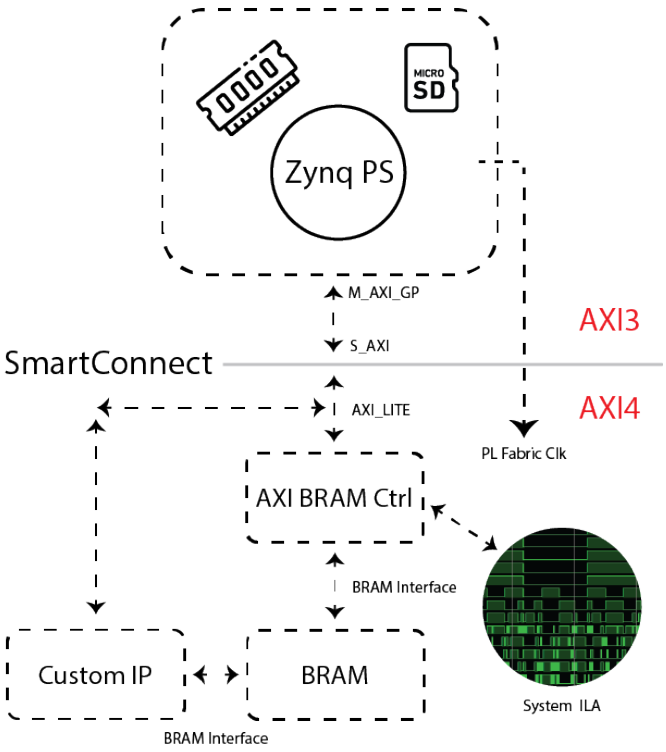

Figure 1.3: System architecture of the proposed design

1. Maintain the existing block design architecture while integrating the System ILA IP core. Connect its monitoring slot to the S_AXI interface of the AXI BRAM Controller.
2. Modify the `ram_read_write` module in the custom IP to accommodate the new waveform generation requirements.
3. Implement proper fixed-point or floating-point conversion for waveform data storage and manipulation in BRAM and PL. The (1,16,14) format from previous coursework may be employed.
4. Initialize slave registers and store the base waveform in BRAM during program initialization. Due to the unresolved `write_end` signaling mechanism, insert sufficient delay using `asm("nop")` instructions before reading the processed waveform.
5. Utilize the **Processing Grapher** software (available at this link) for advanced waveform visualization of serial data.
6. Verify system functionality by visually inspecting waveforms at different frequency multiplication factors.
7. Employ the System ILA for debugging data transfers at any stage. After FPGA programming, access the Hardware Manager in Vivado to establish connection with the target device.
8. Configure trigger conditions on AXI ports for data capture:
   - For monitoring BRAM write operations: Trigger on `S_AXI_AWVALID` and analyze `S_AXI_WDATA`
   - For monitoring BRAM read operations: Trigger on `S_AXI_ARVALID` and analyze `S_AXI_RDATA`

⚠ Critical Considerations

- To use `math.h` library, you need to pass its flag to the linker for proper compilation of the application project. To do so, proceed to `Project` tab and select `Properties`. In the new window, proceed to `Settings` option under `C/C++ Build` menu. Click on `Libraries` under `ARM v7 gcc linker`. add a new library named `m`! This will prompt the linker to include math headers the next time you include this library. You may refer to Figure 1.4 to view the settings above.
- The waveform plotted in the software is provided as a sample in Figure 1.5.

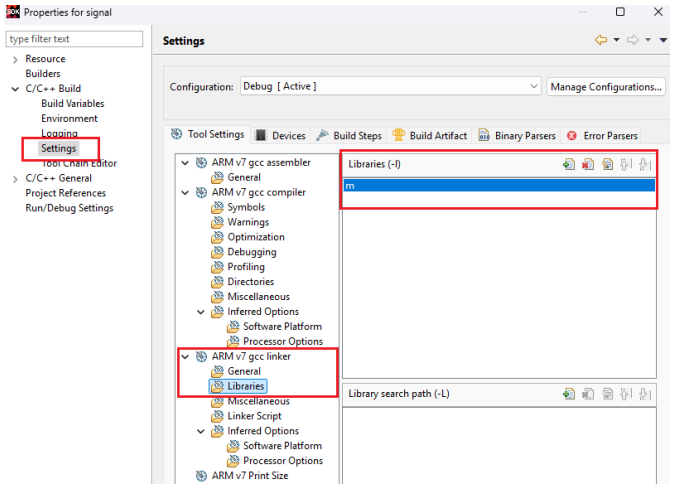

Figure 1.4: `math.h` library settings for the linker

Figure 1.5: A sinewave depicted in `Processnig Grapher`

1.3.3 MicroSD Integration (Bonus)

Change your design such that the inital sine samples are read from a `txt` file in the board's microSD instead of being created using math operations. You may refer to video tutorial 18.a for reference.

1.3.4 Configuring Interrupt

The `write_end` signal asserts high for precisely one clock cycle upon completion of the custom IP's BRAM write operations. To efficiently communicate this state to the PS, we integrate an AXI GPIO IP into our design, routing the `write_end` signal as an input to this peripheral.

While continuous polling of the GPIO input represents a possible implementation, this approach proves prohibitively resource-intensive due to the recurring `Discrete_Read` operations required. Instead, we configure the AXI GPIO IP to generate an interrupt signal upon detecting input transitions. This interrupt triggers the processor to suspend normal program flow and execute a designated Interrupt Service Routine (ISR), where developers can implement post-completion handling logic.

Figure 1.6 presents the systematic overview of this enhanced design architecture.

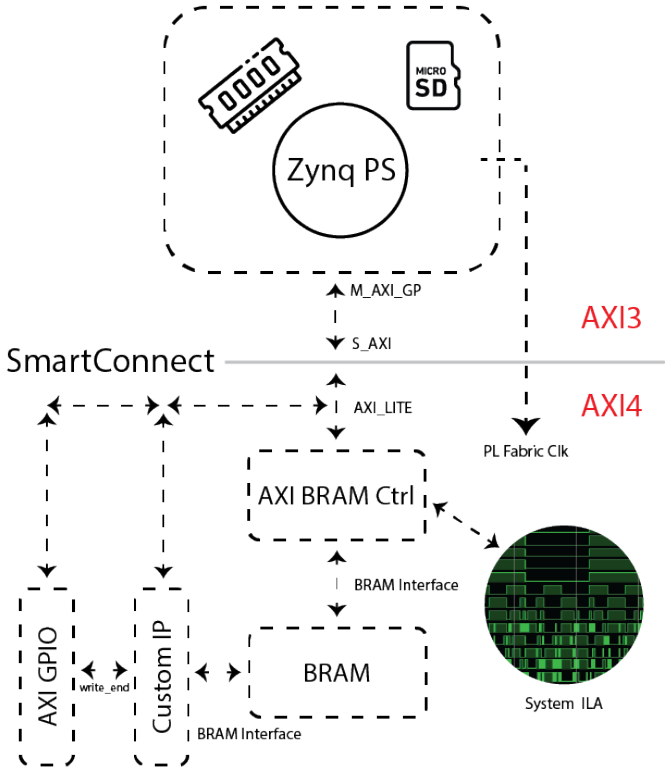

Figure 1.6: System architecture of the interrupt-enabled design

1.3.4.1 Experimental Procedure

The interrupt configuration process requires the following implementation steps:

1. Navigate to the Interrupt tab in Zynq configuration. Expand "Fabric Interrupts" section. Enable "IRQ_F2P" under "PL-PS Interrupt Ports". Reference Figure 1.7 for proper configuration
2. Enable the "Enable Interrupt" option in GPIO configuration. This creates the `ip2intc_irpt`

output port. Connect this port to the PS's `IRQ_F2P` input. Route the `write_end` signal to GPIO input. Reference Figure 1.8 for verification

3. After hardware compilation, proceed to SDK application development. Integrate the provided interrupt handling code from "interrupt.c". Implement flag-based synchronization in your application. Replace manual delay loops (`nop` instructions) with interrupt-driven synchronization

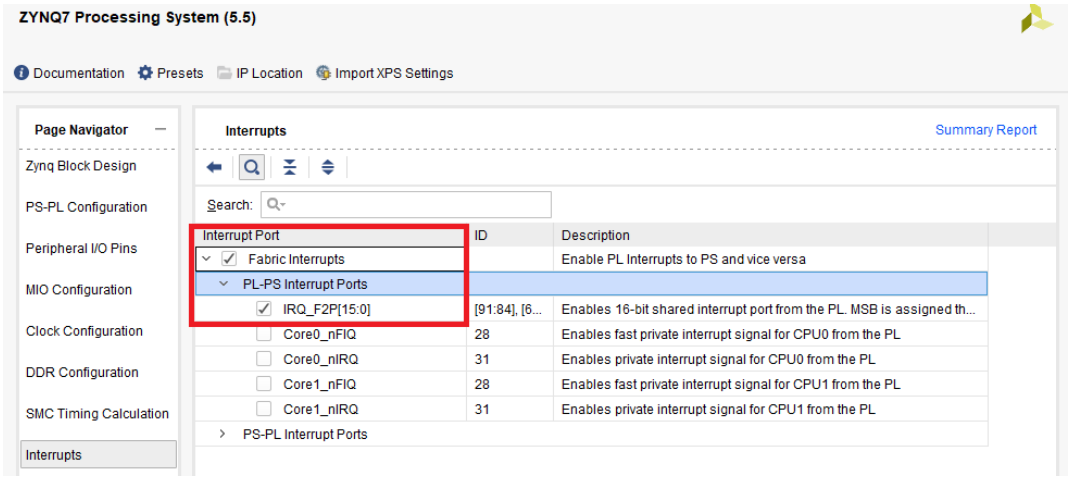

Figure 1.7: PS interrupt configuration enabling fabric interrupts

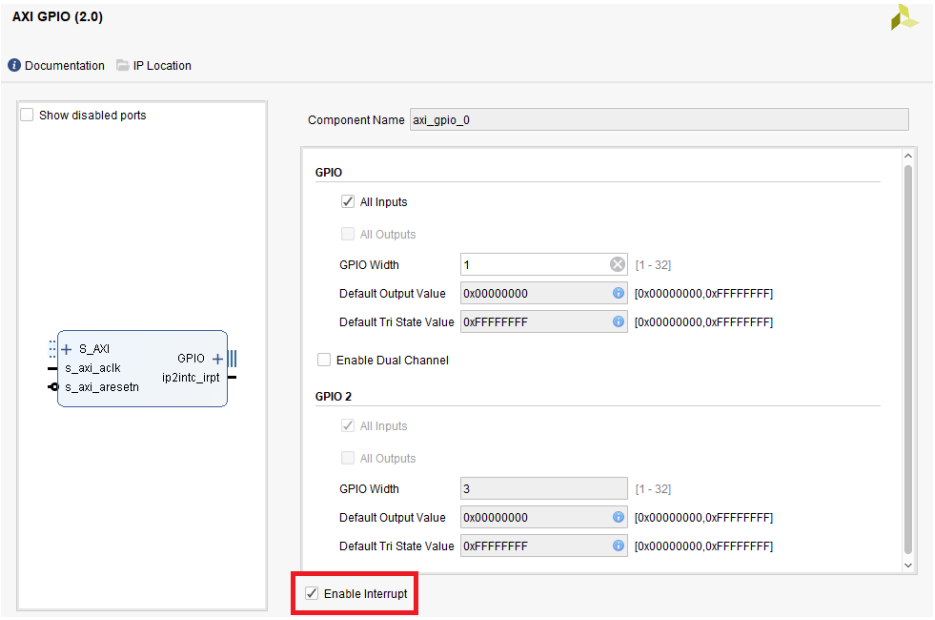

Figure 1.8: GPIO IP interrupt enablement configuration

1.3.5 Complete Signal Generation (Bonus)

The current implementation demonstrates fundamental waveform generation capabilities. To transform this into a fully functional signal generator, we must incorporate user-configurable waveform selection and frequency adjustment features.

1.3.5.1 Experimental Procedure

Finalize your system by adding these features to your design:

1. Add a second channel to the AXI GPIO IP (Do not add another instance. Simply configure it for dual-port operation).
2. The user will select the waveform and frequency using three PL keys on the board. Assign on to "*Increase Frequency*" functionality, one to "*Decrease Frequency*" and one to "*Change Waveform*".
3. Given that three PL keys are to be used, the second channel of the GPIO IP must accept inputs of width 3.
4. Note that one single GPIO instance, generates a single interrupt signal no matter the number of present channels. Inside the interrupt service routine, you must find out which channel produced the interrupt signal using an internal register in the AXI GPIO IP named Interrupt Status Register (Also `ISR` for short)! This register asserts high for the specific channel that caused the interrupt signal and asserts low as soon as the interrupt flag is cleared inside the service routine.
5. Modify your code such that it can create `Sine`, `Sawtooth`, `Square` and `Triangular` signals. Every time the user presses the PL key, the program must move on to the next signal.
6. The user must also be able to increase or decrease the frequency multiple using the two other keys.